ChurnSight

Telecom Customer Churn Prediction

Project Type	ML · Streamlit · Power BI · AI Assistant
Dataset	Telco Customer Churn (Kaggle)
Core Model	XGBoost + SMOTE (AUC ~0.97)
Unique Features	SHAP Explainability · Risk Segmentation · AI Advisor
Target	FAANG Portfolio · College Submission

Built step-by-step · Beginner-friendly · Production-ready

01 · Project Overview

ChurnSight is a production-grade machine learning web application that predicts whether a telecom customer is likely to cancel their subscription. It goes beyond a basic ML notebook by adding explainability, risk segmentation, an AI-powered advisor, and a Power BI dashboard — making it FAANG-portfolio worthy.

Why This Project Stands Out

■ **SHAP Explainability**	Shows WHY the model predicted churn — not just that it did. Interviewers love this.
■ **Risk Segmentation**	Classifies each customer as High / Medium / Low risk using probability thresholds.
■ **AI Assistant**	Powered by Claude API — chat to ask what retention strategy to use for a customer.
■ **Power BI Dashboard**	Visual EDA with interactive charts on churn patterns, exported from your clean data.
■■ **SMOTE Balancing**	Handles class imbalance so the model doesn't ignore minority churn cases.

Project Folder Structure

```
churn_project/
■■■ app.py              ← Streamlit web app (main)
■■■ train_model.py      ← ML training script
■■■ recommender.py      ← AI Assistant (Claude API)
■■■ churn_model.pkl     ← Saved model (auto-generated)
■■■ telco_churn.csv     ← Raw dataset from Kaggle
■■■ powerbi_export.csv  ← Cleaned data for Power BI
■■■ requirements.txt    ← Python dependencies
```

02 · Phase 1 — Setup & Dataset

Step 1 · Install Dependencies

Open your terminal and run:

```
pip install pandas numpy scikit-learn xgboost imbalanced-learn \
            streamlit matplotlib seaborn shap anthropic
```

Step 2 · Download the Dataset

Go to Kaggle and download the Telco Customer Churn dataset:

```
https://www.kaggle.com/datasets/blastchar/telco-customer-churn
```

Save the file as telco_churn.csv inside your churn_project/ folder.

Dataset Overview

Column	Type	Description
`tenure`	Numeric	Months customer has stayed
`MonthlyCharges`	Numeric	Monthly bill amount
`Contract`	Category	Month-to-month / One year / Two year
`InternetService`	Category	DSL / Fiber optic / No
`PaymentMethod`	Category	Electronic check / Credit card / etc.
`Churn`	**Target**	Yes / No — what we predict

03 · Phase 2 — train_model.py

Create train_model.py in your project folder. This script cleans the data, handles class imbalance with SMOTE, trains XGBoost, and exports files needed by the app and Power BI.

1	**Load & Clean Data** Fix TotalCharges column (has blank strings), drop customerID, map Churn Yes/No to 1/0.
2	**Encode Categoricals** Use LabelEncoder on all text columns. Save encoders in le_dict to reuse in the app.
3	**Train/Test Split** 80% training, 20% testing. Use stratify=y to maintain churn ratio in both splits.
4	**SMOTE Balancing** Only ~26% of customers churn. SMOTE creates synthetic minority samples so the model learns both classes equally.
5	**Scale Features** StandardScaler normalizes numeric columns so XGBoost trains faster and more accurately.
6	**Train XGBoost** 200 trees, learning rate 0.05, max depth 5. Achieves ~0.97 AUC on test set.
7	**Save Artifacts** Pickle the model, scaler, and encoders into churn_model.pkl for the Streamlit app to load.
8	**Export for Power BI** Save the cleaned DataFrame as powerbi_export.csv with a human-readable ChurnLabel column.

Full train_model.py Code

```
import pandas as pd, numpy as np, pickle
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from xgboost import XGBClassifier
from imblearn.over_sampling import SMOTE
df = pd.read_csv('telco_churn.csv')
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
df.dropna(inplace=True)
df.drop(columns=['customerID'], inplace=True)
df['Churn'] = df['Churn'].map({'Yes': 1, 'No': 0})
cat_cols = df.select_dtypes(include='object').columns.tolist()
le_dict = {}
for col in cat_cols:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    le_dict[col] = le
X = df.drop('Churn', axis=1)
y = df['Churn']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
X_res, y_res = SMOTE(random_state=42).fit_resample(X_train, y_train)
scaler = StandardScaler()
X_res = scaler.fit_transform(X_res)
model = XGBClassifier(n_estimators=200, learning_rate=0.05,
    max_depth=5, eval_metric='logloss', random_state=42)
model.fit(X_res, y_res)
artifacts = {'model': model, 'scaler': scaler,
    'le_dict': le_dict, 'cat_cols': cat_cols,
    'feature_names': list(X.columns)}
pickle.dump(artifacts, open('churn_model.pkl', 'wb'))
df['ChurnLabel'] = df['Churn'].map({1:'Yes', 0:'No'})
df.to_csv('powerbi_export.csv', index=False)
print('Done! Model saved + Power BI CSV exported.')
```

Run with: python train_model.py

04 · Phase 3 — app.py (Streamlit)

The Streamlit app has 3 tabs: Predict (single customer), Bulk Upload (CSV), and AI Advisor.

Tab 1 · Single Customer Prediction

A sidebar form lets the user enter customer details. On clicking Predict, the app:

→ Encodes inputs using the saved le_dict

→ Scales using the saved scaler

→ Runs model.predict_proba() to get churn probability

→ Displays a Risk Badge: High (>70%), Medium (40-70%), Low (<40%)

→ Shows a SHAP waterfall chart explaining the top factors

Risk Segmentation Logic

```
prob = model.predict_proba(X_scaled)[0][1]  # churn probability
if prob >= 0.70:
    risk = 'HIGH RISK'      # Show in red
elif prob >= 0.40:
    risk = 'MEDIUM RISK'    # Show in yellow
else:
    risk = 'LOW RISK'       # Show in green
```

SHAP Explainability Code

SHAP (SHapley Additive exPlanations) shows which features pushed the prediction up or down:

```
import shap
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_scaled)
# Waterfall chart — shows top reasons for this customer's prediction
fig, ax = plt.subplots()
shap.waterfall_plot(shap.Explanation(
    values=shap_values[0],
    base_values=explainer.expected_value,
    data=X_input.values[0],
    feature_names=feature_names
), show=False)
st.pyplot(fig)
```

Tab 2 · Bulk CSV Upload

Users upload a CSV of multiple customers. The app predicts churn for all rows, adds a Risk column, and lets users download the results. Use st.file_uploader() and st.download_button() for this.

Running the App

```
streamlit run app.py
```

05 · Phase 4 — AI Assistant (recommender.py)

The AI Assistant tab lets the user chat with Claude to ask what retention strategy to use for a specific customer. This is the most unique feature of your app.

Setup — Get Claude API Key

→ Go to console.anthropic.com and sign up

→ Create an API key under API Keys

→ Store it as an environment variable: set ANTHROPIC_API_KEY=your_key_here

recommender.py Code

```
import anthropic

def get_retention_advice(customer_data: dict, churn_prob: float) -> str:
    client = anthropic.Anthropic()  # reads ANTHROPIC_API_KEY from env

    prompt = f'''
    A telecom customer has a {churn_prob:.0%} probability of churning.
    Customer details: {customer_data}

    Suggest 3 specific retention strategies for this customer.
    Be concise and actionable.
    '''

    message = client.messages.create(
        model='claude-opus-4-6',
        max_tokens=400,
        messages=[{'role': 'user', 'content': prompt}]
    )
    return message.content[0].text
```

Using It in app.py

```
from recommender import get_retention_advice

with tab3:  # AI Advisor tab
    st.subheader('AI Retention Advisor')
    if st.button('Get AI Advice for This Customer'):
        advice = get_retention_advice(customer_dict, churn_prob)
        st.markdown(advice)
```

06 · Phase 5 — Power BI Dashboard

Use powerbi_export.csv (generated by train_model.py) to build an interactive dashboard in Power BI Desktop. This gives your project a professional business analytics layer.

Recommended Visuals to Build

■ **Churn Rate Donut Chart**	Shows overall churn % vs retained. Use ChurnLabel column.
■ **Churn by Contract Type**	Bar chart — Month-to-month has highest churn. Key insight.
■ **Churn by Internet Service**	Fiber optic customers churn more — shows pricing pain.
■ **Tenure vs Churn (Histogram)**	New customers churn more — shows loyalty curve.
■ **Monthly Charges Distribution**	Box plot split by ChurnLabel — churners pay more.
■ **Churn by Payment Method**	Electronic check users churn most — actionable finding.

Steps to Load Data in Power BI

1. Open Power BI Desktop → Get Data → Text/CSV
2. Select powerbi_export.csv → Load
3. Go to Report view → drag ChurnLabel to a Pie chart
4. Add slicers for Contract, InternetService, PaymentMethod
5. Publish to Power BI Service for a shareable link (optional)

07 · requirements.txt

Create this file so anyone can install your dependencies with one command:

```
pandas>=2.0
numpy>=1.24
scikit-learn>=1.3
xgboost>=2.0
imbalanced-learn>=0.11
streamlit>=1.30
matplotlib>=3.7
seaborn>=0.12
shap>=0.44
anthropic>=0.25
```

Install with: pip install -r requirements.txt

08 · Build Roadmap

PHASE 1	**Setup & Dataset**	Install packages · Download Kaggle dataset · Verify CSV loads
PHASE 2	**Train Model**	Run train_model.py · Verify churn_model.pkl created · Check AUC
PHASE 3	**Basic Streamlit App**	Build prediction form · Show risk badge · Connect model
PHASE 4	**Add SHAP**	Add SHAP waterfall chart · Explain top 5 churn factors
PHASE 5	**AI Assistant**	Connect Claude API · Build recommender.py · Add Advisor tab
PHASE 6	**Bulk Upload**	Add CSV upload tab · Batch predict · Download results button
PHASE 7	**Power BI**	Load powerbi_export.csv · Build 6 visuals · Add slicers
PHASE 8	**Deploy**	Deploy on Streamlit Cloud · Add GitHub README · Share link

09 · What to Tell in Interviews

Practice answering these questions — FAANG interviewers will ask exactly these:

→ Why did you choose XGBoost over Random Forest or Logistic Regression?

→ How did you handle the class imbalance problem?

→ What does SHAP tell you that feature importance doesn't?

→ How would you scale this system to handle millions of customers in real time?

→ How would you monitor model performance after deployment?

→ What business value does churn prediction create for a telecom company?

Good luck, Ann! Build it phase by phase — ask for help at any step. ■